

2.680 Labs 02 and 03 - MOOS Intro and Programming

Introductory MOOS concepts and MOOS programming lab material.

Included MIT MOOS-IvP PDF sources:

- lab_class_02_intro_moos.pdf
- lab_class_03_moos_prog.pdf

Lab 2 - Introduction to MOOS

2.680 Unmanned Marine Vehicle Autonomy, Sensing and Communications



February 5th, 2026

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1	Overview and Objectives	3
1.1	Preliminaries	3
1.2	MOOS vs. MOOS-IvP	3
1.3	More MOOS / MOOS-IvP Resources	4
1.4	The MOOS Architecture	4
1.5	Launching the MOOSDB	6
1.6	Scoping the MOOSDB	8
1.7	Poking the MOOSDB	13
1.8	Launching a Mission with pAntler	16
1.9	Scripted Pokes to the MOOSDB	18
2	A Simple Example with pXRelay	22
2.1	Basic pXRelay Usage	22
2.2	A Simple Example with pXRelay	22
3	First Experiments in Modifying MOOS App Code	24
3.1	Download a Template Tree for Extending MOOS	24
3.2	Building Your Tree	24
3.3	Exercises in Modifying the pXRelayTest Code	24

1 Overview and Objectives

This lab will introduce MOOS to new users. It assumes nothing regarding MOOS background. The goals of this lab are to (a) understand the publish-subscribe architecture, (b) get comfortable launching and interacting with the MOOSDB, (c) understand how to generate scripted interactions with the MOOSDB, (d) understand how the logger operates and basic tools for examining log files.

- MOOS Preliminaries: MOOS vs. MOOS-IvP, the MOOS Architecture
- Launching, Scoping, and Poking the MOOSDB
- Launching a Mission with pAntler
- Scripted Pokes to the MOOSDB
- A Simple Example with pXRelayTest
- Modify the pXRelayTest Code

1.1 Preliminaries

This lab assumes you have a working MOOS-IvP tree checked out and built on your computer. To verify this make sure that the following executables are built and findable in your shell path:

```
$ which MOOSDB
/Users/you/moos-ivp/bin/MOOSDB      (or /home/you/moos-ivp/bin/MOOSDB on Linux)
$ which pHelmIvP
/Users/you/moos-ivp/bin/pHelmIvP
```

If unsuccessful with the above, return to the steps in Lab 1:

http://oceanai.mit.edu/ivpman/labs/machine_setup

1.2 MOOS vs. MOOS-IvP

What is the relationship between MOOS and MOOS-IvP? MOOS-IvP is a superset of MOOS. The additional components include another architecture, the IvP Helm behavior-based architecture, and several additional MOOS applications. This is the nested repository concept depicted in Figure 1.

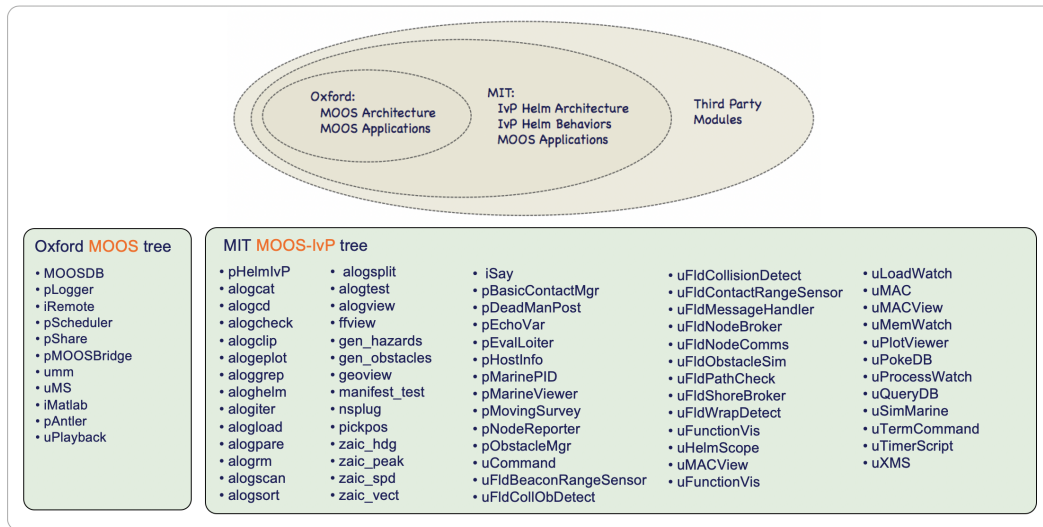


Figure 1: **Nested Repositories:** The MOOS-IvP tree contains the Oxford MOOS tree and additional modules from MIT including the Helm architecture, Helm behaviors and further MOOS applications.

1.3 More MOOS / MOOS-IvP Resources

We will only just touch the MOOS basics today. A few further resources are worth mentioning for following up this lab with your own exploration.

- The 2.680 lecture covers much of the material for today's lab:
http://oceanai.mit.edu/2.680/docs/2.680-03-intro_to_moos_2025.pdf
- The "Very Brief Overview of MOOS" page on the course documentation page:
http://oceanai.mit.edu/ivpman/chap/moos_overview
- Follow the links to the documentation on the Oxford MOOS github website.
<https://github.com/themoos>

1.4 The MOOS Architecture

The main idea explored today is that MOOS is a publish-subscribe architecture. A single **MOOSDB** serves multiple MOOS applications by essentially handling the mail published and subscribed for by each app. A MOOS *community* is a collection of applications connected to a single **MOOSDB**.

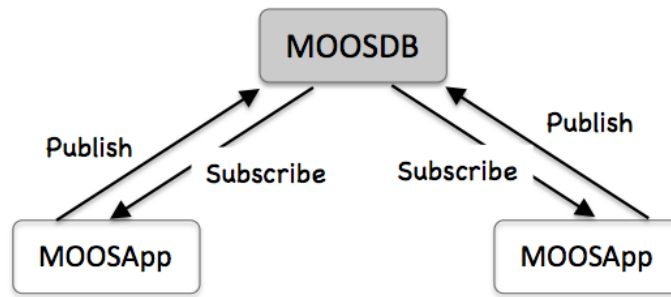


Figure 2: **The MOOS Architecture:** MOOS is a publish-subscribe architecture. The **MOOSDB** serves a number of clients, handling mail for each client as new information is posted. A MOOSDB with connected clients constitutes a *MOOS community*. There may be multiple MOOS communities on a single machine, and a single MOOS community may be distributed over more than one machine.

For typical autonomous vehicle implementations, there is a MOOS community on board each vehicle. When simulating multiple vehicles on a single machine, there is also a single community associated with each vehicle. A *MOOS community* consists of a single **MOOSDB** with one or more connected clients (i.e. apps). The communications discussed in today's lab concern how a single app communicates with another app via the **MOOSDB** in the publish-subscribe architecture. Later labs will address how vehicles communicate with each other, essentially bridging two or more MOOS communities with one another.

For today, the focus is on the **MOOSDB** and connected applications. The **MOOSDB**, unlike an actual database, does not contain a full history of information that has passed through it. At most, it stores the latest value for any given MOOS variable published to the **MOOSDB**. When a new app connects to the **MOOSDB** it must register for the mail it needs. On startup, an app can expect to get a mail message containing the latest value for any variable it registers for, even if that mail reflects a posting to the **MOOSDB** long ago. Anything happening prior to that will be unknown to the newly connected app.

1.5 Launching the MOOSDB

Here we describe how to launch the **MOOSDB** from the perspective of the first-time user. The **MOOSDB** application is a server that runs on the robot or unmanned vehicle computer, or simply on your laptop during simulation. It may be launched from the command line, assuming it is in your path. Two minimalist methods are described here, starting with the most bare-bones.

1.5.1 A Bare-Bones Launching of the MOOSDB

In a bare-bones manner, the **MOOSDB** may be launched from the command line without any arguments. Normally the **MOOSDB** needs to know at least two pieces of configuration information, (a) the machine IP address on which to run, and (b) the port number on which to serve clients.

(Recall an IP address is comprised of four numbers in the range of [0,255] separated by a period. IP addresses are used for finding a particular machine on the Internet. For example, the oceanai class server resides at 18.18.38.22. The term "localhost" acts as a stand-in IP address when referring to one's own machine. The port number refers to a channel of sorts on a computer. It's fair to think of the IP address like the street address of an apartment building, and the port number as the unit number within the building where a particular resident resides.)

When launching the **MOOSDB** with no command line arguments, it will default to running on the localhost and port 9000:

```
$ MOOSDB
----- MOOSDB V10 -----
Hosting community      "#1"
Name look up is       off
Asynchronous support is on
Connect to this server on port 9000
-----
network performance data published on localhost:9020
listen with "nc -u -lk 9020"
```

At this point the **MOOSDB** is running on the local machine, serving clients on port 9000. A few variables are already being published, by the **MOOSDB** itself. You can open a scope with **uXMS** in another terminal window:

```
$ uXMS DB_CLIENTS DB_TIME DB_UPTIME --serverhost=localhost --serverport=9000

=====
uXMS_581                                     0/0(154)
=====
VarName      (S)ource      (T)ime      (C)   VarValue (SCOPING:EVENTS)
-----
DB_CLIENTS   MOOSDB_#1    38.14      "uXMS_581,"
DB_TIME      MOOSDB_#1    38.14      1387380731.414707
DB_UPTIME    MOOSDB_#1    38.14      39.00859
```

The Source and Time columns may be expanded by hitting the "s" and "t" keys respectively after

it launches, or you can add `--show=source,time` as a command line argument to `uXMS`, to launch with these two columns expanded.

Alternatively you can scope with the `umm` tool:

```
$ umm --spy
```

The `umm` tool is part of the MOOS project at Oxford, while `uXMS` tool is part of the MOOS-IvP project at MIT. Overlap between toolset functions can often be found.

1.5.2 A More Civilized Launching of the MOOSDB

Virtually all MOOS applications are launched with a "mission configuration" file, a.k.a. a "dot-moos" file. The below mission file, `moosdb_alpha.moos`, provides the minimal configuration parameters the `MOOSDB` likes to see upon starting.

```
// (wget http://oceanai.mit.edu/2.680/examples/moosdb_alpha.moos)
ServerHost = localhost
ServerPort = 9000
Community  = alpha
```

Passing the `moosdb_alpha.moos` file as a command line argument produces the below output.

```
$ MOOSDB moosdb_alpha.moos
----- MOOSDB V10 -----
Hosting community      "alpha"
Name look up is        off
Asynchronous support is on
Connect to this server on port 9000
-----
network performance data published on localhost:9020
listen with "nc -u -lk 9020"
```

Try this. Either copy and paste the above `.moos` file, or use `wget` as shown on the first line above in the example file. If you haven't installed `wget` or similar (e.g., `curl`), you should pause here and do that.

```
$ sudo apt install wget      (On Linux)
$ brew install wget          (On MacOS using Homebrew)
$ sudo port install wget     (On MacOS using MacPorts)
```

1.6 Scoping the MOOSDB

A MOOS *scope* is a tool for examining the state of the **MOOSDB**. The **MOOSDB** does not keep a history of prior values for a given variable, but rather just the most recent value posted. This means that the *state* of the **MOOSDB** may be regarded as the set of current MOOS variables, their values, and who made the last posting to the variable and when. Scoping allows a view into the current state, or even recent history, of the **MOOSDB**. If a scope shows a recent history, it is because the scope tool itself retains that history, since the MOOSDD does not.

There are multiple tools for scoping the DB, each providing conveniences of one kind or another. Here we describe the **uXMS**, **uMS**, and **pRealm** tools, the three favorites of our own lab.

More info on these two tools can be found at:

- uXMS: <http://oceanai.mit.edu/ivpman/apps/uXMS>
- pRealm: <http://oceanai.mit.edu/ivpman/apps/pRealm>

You can also navigate to the above two web pages by launching the app with the `--web` or `-w` option.

```
$ uXMS --web
$ pRealm -w
```

Most MOOS-IvP distributed apps support this command-line option.

1.6.1 Scoping the MOOSDB with uXMS

Your goals in this part are:

1. Open a terminal window and launch the **MOOSDB** as done at the end of the previous exercise:

```
$ MOOSDB
----- MOOSDB V10 -----
Hosting community      "#1"
Name look up is        off
Asynchronous support is on
Connect to this server on port 9000
-----
network performance data published on localhost:9020
listen with "nc -u -lk 9020"
```

The **MOOSDB** is normally launched with a mission file specifying the **ServerHost** and **ServerPort** parameters. When launched from the command line as above with no command line arguments, these two parameters default to `localhost` and `9000`.

2. Open a second terminal window and launch **uXMS**, passing it the `--all` command line switch. Just hit `ENTER` when prompted for the IP address and Port number, accepting the defaults of `localhost` and `9000`. It should look something like:

```

$ uXMS --all
Enter IP address: [localhost]
Enter Port number: [9000]
*****
* uXMS_632 starting ...
*****
uXMS_632 is Running:
|-Baseline AppTick @ 5.0 Hz
|--Comms is Full Duplex and Asynchronous
-Iterate Mode 0 :
|-Regular iterate and message delivery at 5 Hz

```

After the above initial standard MOOSApp output, you should see a sequence of **uXMS** reports similar to:

```

=====
uXMS_443                                     0/0(31)
=====
VarName          (S) (T) (C)  VarValue (SCOPING:EVENTS)
-----
DB_CLIENTS              "uXMS_443,"
DB_EVENT                "connected=uXMS_443"
DB_QOS                  "uXMS_443=0.448942:0.573874:0.364065:0.490189,"
DB_TIME                 1424028179.877422
DB_UPTIME               16.741557
UXMS_443_ITER_GAP       1.019045
UXMS_443_ITER_LEN       0.00024
-- displaying all variables --

```

Notice the number in parentheses on the second line is incrementing. This indicates that the report has been refreshed to your terminal. If you launched as above, the scope should come up in a mode that refreshes the report any time a scoped variable changes values. In this case, the **MOOSDB** is updating **DB_TIME** and **DB_UPTIME** about once per second. By default, **uXMS** only scopes on the variables named on the command line. In the above case, the **--all** option was used to tell **uXMS** to scope on all variables known to the **MOOSDB**.

The five variables shown beginning with **DB_** are all published by the **MOOSDB**. The user may choose whether or not to show the variable (S)ource, (T)ime of post, or (C)ommunity from where the post was made. A key feature of **uXMS** vs. **uMS** is the ability to specify on the command-line exactly which subset of variables to scope, possibly with color-coding. This is helpful when there are hundreds of variables in the DB.

1.6.2 More Suggested Tinkering with uXMS

Try a few other things:

1. Hit the 'h' key to see some keyboard interaction options that are available anytime the scope is running. Hit 'h' again any time to return to the previous mode.
2. Hit the space-bar to pause the stream of reports. This is useful if numbers are changing rapidly and you just need to take a close look at something. Return to the previous mode by hitting 'e'.

3. Hit the 's' key to expand the (S)ource column. This column tells you which app made the last posting. Try the same for the (T)ime and (C)ommunity columns.
4. The whole purpose of a scope is to give you the key information you're looking for, without needing to sift through a lot of unwanted information, with as little effort as possible. In this step we'll pretend to be interested in focusing our attention on the `DB_UPTIME` variable. Try launching `uXMS` with an additional command line argument:

```
$ uXMS --all --colormap=DB_UPTIME,blue
```

This may seem unnecessary when there are only three variables, but in real applications there may be hundreds of variables. In fact, the variable you're looking for may have scrolled off the window!

5. A similar way to focus on a single variable is to only scope on the one variable we're looking for:

```
$ uXMS DB_UPTIME
```

6. `uXMS` only shows you the current snapshot of the variables in the `MOOSDB`. What if you would like to see how a variable is changing? In our case, we know how the `DB_UPTIME` variable is changing, but for the sake of showing this feature, try:

```
$ uXMS --history=DB_UPTIME
```

1.6.3 Scoping with uMS

The `uMS` scope is a graphical MOOS scope, often preferred by those inclined to like GUIs vs. command line tools. It has some other advantages over `uXMS` as well.

Your goals in this part are:

1. If you don't still have a `MOOSDB` running, open a terminal window and launch the `MOOSDB` as done previously:

```
$ MOOSDB
```

2. Open a second terminal window and launch `uMS`, passing it the same mission file as an argument:

```
$ uMS
```

You should see a window open and, after clicking on the `Connect` button, you should see something similar to:

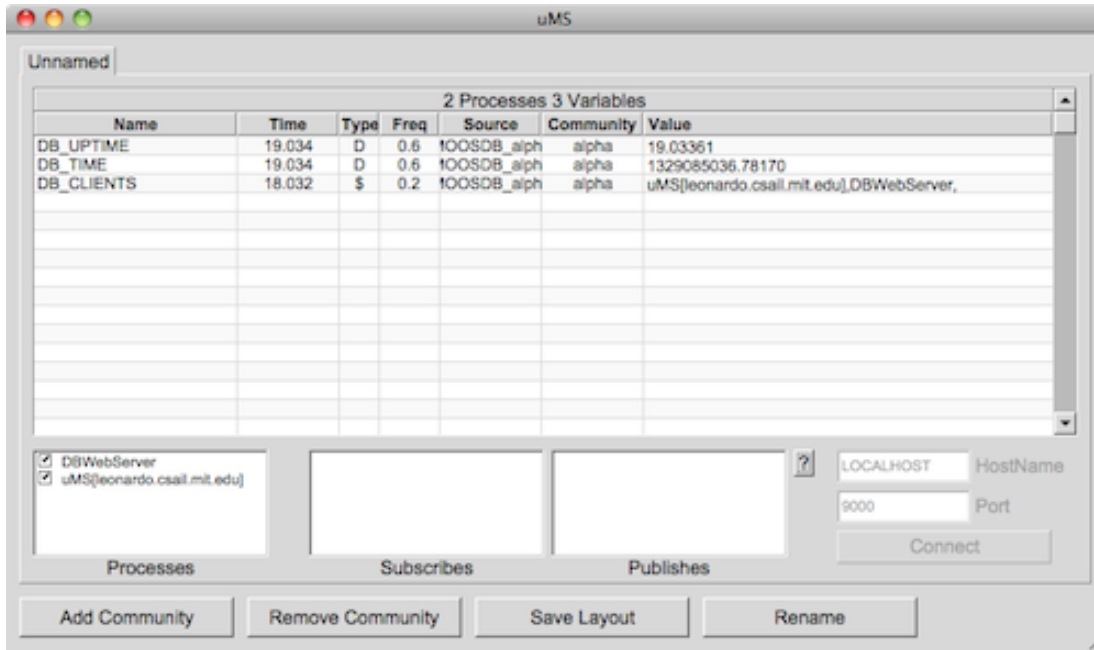


Figure 3: The uMS MOOS Scope

1.6.4 Pros/Cons of uXMS vs uMS

The choice of **uXMS** vs **uMS** is often just a matter of taste. A couple of differences are noteworthy however.

- **uMS** allows for connections to multiple MOOSDBs, on perhaps multiple vehicles, simultaneously. The user may select the vehicle with the tab at the top of the screen.
- **uXMS** allows the user to scope on as few as one single variable, or to name the variable scope list explicitly. **uMS** scopes on all variables all the time, with a few mechanisms for reducing the scope list based on process name.
- **uXMS** may be a better choice if one is scoping on a remote **MOOSDB**, perhaps on a robot with a poor connection. It is a low-bandwidth client compared to **uMS**. If running on a remote terminal, its bandwidth back to the user is zero in the paused mode.
- **uXMS** has provisions for at least limited scoping on a variable history.
- **uXMS** will display a variable's "auxiliary source" information. This is a secondary field associated with each posting describing the source of the posting. This is key when using the IvPHelm. Variables posted by helm behaviors will have a source of **pHelmIvP** and an auxiliary source showing the behavior responsible for the posting.

1.6.5 Scoping with pRealm

Realmcasting is a feature first introduced in 2021. It complements Appcasting, which is the default set of windows open to the left of the GUI when running **pMarineViewer** in say the Alpha mission. If you are running a mission with **pMarineViewer**, it may be the only scoping tool you need. You can scope on selected vehicle (MOOS community), and selected app, as shown in Figure 4.

1.7 Poking the MOOSDB

Poking refers to the idea of publishing a variable-value pair to the **MOOSDB**. Many apps publish to the **MOOSDB** during the course of normal operation. Poking implies a publication that perhaps was not planned, or outside the normal mode of business. It is often very useful for debugging. Here we describe the **uPokeDB** tool.

Where to get more information:

- **uPokeDB**: <http://oceanai.mit.edu/ivpman/apps/uPokeDB>

Or simply:

```
$ uPokeDB --web
```

1.7.1 Poking the MOOSDB with uPokeDB

uPokeDB is a command-line tool for poking the **MOOSDB** with one or more variable-value pairs. Poking the **MOOSDB** requires knowing where the **MOOSDB** is running in terms of its IP address, (**ServerHost** parameter), and port number, (**ServerPort** parameter). These may be specified on the command line to **uPokeDB**, but for our purposes here we assume the existence of a mission file, **alpha.moos** with this information:

```
// (wget http://oceanai.mit.edu/2.680/examples/alpha.moos)
ServerHost = localhost
ServerPort = 9000
Community  = alpha
```

Your goals in this part are:

1. Open two terminal windows and launch the **MOOSDB** and **uXMS** as done previously:

```
$ MOOSDB alpha.moos
$ uXMS alpha.moos --all
```

Now open a third terminal window for poking the **MOOSDB** as follows:

```
$ uPokeDB DEPLOY=true SPEED=2 alpha.moos
```

Note the two new variables, **DEPLOY** and **SPEED**, appearing in the **uXMS** window. It should look something like:

=====				
uXMS_655				
				0/0(204)
=====				
VarName	(S)	(T)	(C)	VarValue (SCOPING:EVENTS)

DB_CLIENTS				"uXMS_655,"
DB_TIME				1386249435.276804
DB_UPTIME				46.213629
DEPLOY				"true"
SPEED				2

- Note the variable values in **uXMS**. **DEPLOY** has the value "true" with double quotes, indicating that it is a string. The variable **SPEED** is of type double, indicated by the lack of quotes. The types were inferred by **uPokeDB** by heuristically checking whether the arguments are numerical or not. But sometimes you *do* want to publish a string with a numerical value. Try posting the variable **HEIGHT** with the string value of "192", noting the colon-equals instead of equals:

```
$ uPokeDB HEIGHT:=192 alpha.moos
```

Note the new variable, **HEIGHT**, appearing in the **uXMS** window. It should look something like the below output, where in this case, the (S)ource column is expanded to show the source of the postings.

=====				
uXMS_655				
				0/0(347)
=====				
VarName	(S)ource	(T)	(C)	VarValue (SCOPING:EVENTS)

DB_CLIENTS	MOOSDB_alpha			"uXMS_655,"
DB_TIME	MOOSDB_alpha			1386250092.847527
DB_UPTIME	MOOSDB_alpha			703.784353
DEPLOY	uPokeDB			"true"
HEIGHT	uPokeDB			"192"
SPEED	uPokeDB			2

1.7.2 Further things to try Using uPokeDB

Here's some other things to consider and try:

- Trying poking the **DEPLOY** variable to the **MOOSDB** a second time, this time with:

```
$ uPokeDB DEPLOY=100 alpha.moos
```

Does the value of **DEPLOY** change? If not, why not?

- Create a simple script of pokes on the command line as follows:

```
$ uPokeDB APPLES=1 alpha.moos; sleep 5; uPokeDB APPLES=2 alpha.moos;
```

If you're new to the command line environment, the semicolon above separates successful command line invocations. The **sleep** command is a common shell utility that will simply pause a given number of seconds before completing.

3. Another way to execute the same simple script as above is to store the above three commands in a file named, for example, `myscript`:

```
uPokeDB APPLES=1 alpha.moos  
sleep 5  
uPokeDB APPLES=2 alpha.moos
```

With the above file you can make the two successive pokes to the `MOOSDB`, with five seconds in between, with:

```
$ source myscript
```

There are many other ways of poking the `MOOSDB`. All MOOS apps that publish anything are examples. Of course many MOOS applications publish a fixed set of variables that are not easily changeable without re-coding. But certain apps like `uTimerScript` and `pMarineViewer` have built-in configuration file parameters for poking the `MOOSDB` in user configurable ways.

1.8 Launching a Mission with pAntler

In theory a set of N MOOS applications may be launched from N terminal windows, but this is cumbersome in practice. The **pAntler** tool allows this to be done from a single mission file. In this file, a block of lines declares all the apps to be launched with one invocation of **pAntler**.

Where to get more information:

- **pAntler**: <http://oceanai.mit.edu/ivpman/apps/pAntler>

1.8.1 Basic pAntler Usage

The Antler block is typically the first configuration block in a .moos file, declared with **ProcessConfig = ANTLER** as below. The **MSBetweenLaunches** parameter specifies the number of milliseconds between launching processes. Each line thereafter specifies an app to be launched and whether a dedicated console window should be opened for the application.

```
ProcessConfig = ANTLER
{
  MSBetweenLaunches = 200

  Run = MOOSDB      @ NewConsole = true/false
  Run = AnotherApp  @ NewConsole = true/false
  ...
  Run = AnotherApp  @ NewConsole = true/false
}
```

Further options exist beyond the vanilla launch configuration described above, including (a) the ability to launch a given app under an aliased name, (b) specifying command-line arguments to an app at launch time and more. See the documentation.

1.8.2 An Example: Launching the MOOSDB along with uXMS

In the example below we use **pAntler** to launch the **MOOSDB** and the **uXMS** scope from a single mission file. The user preferences for **uXMS** are provided in its configuration block. Type **uXMS --example** on the command line for further options if you're curious.

Note: in this example, the launch process should result in a new **xterm** window opening. If you are running on GNU/Linux, the **xterm** should be available by default. If you are running MacOS, you may need to install **xterm**. If you type **xterm** on the MacOS Terminal command line and you receive a "command not found" error, then you may need to install **xterm**.

<https://www.xquartz.org>

You may need to log out and log back in for this to take effect.

Your goals in this part are:

1. Create a copy of the example mission file shown in Listing 1 below and save it locally as **db_and_uxms.moos**. (hint: the easiest way to do this is to just invoke the **wget** expression on the top line of this file. This will pull the file down from the server into your current directory.)

The mission may be launched from the command-line with:

```
$ pAntler db_and_uXMS.moos
```

This should open a new console window for **uXMS** displaying the variables posted by the DB, with the (S)ource and (T)ime columns expanded, but not the (C)ommunity column.

2. Modify the **uXMS** configuration block in the `.moos` file to configure **uXMS** to keep a history of the **DB.UPTIME** variable. To see configuration options for **uXMS**, type:

```
$ uXMS --example
```

Once you have launched **uXMS** with the new configuration, type 'z' to toggle in and out of history mode.

3. Modify the `db_and_uXMS.moos` file to launch a new terminal window for the **MOOSDB** in addition to the **uXMS** application.

Listing 1.1: A simple mission file.

```
// (wget http://oceanai.mit.edu/2.680/examples/db_and_uXMS.moos)
ServerHost = localhost
ServerPort = 9000
Community  = alpha

ProcessConfig = ANTLER
{
    MSBetweenLaunches = 200

    Run = MOOSDB      @ NewConsole = false
    Run = uXMS        @ NewConsole = true
}

ProcessConfig = uXMS
{
    AppTick    = 4
    CommsTick  = 4

    VAR        = DB_CLIENTS, DB_UPTIME, DB_TIME
    DISPLAY_SOURCE = true
    DISPLAY_TIME  = true
    COLOR_MAP    = DB_CLIENTS, red
}
```

1.9 Scripted Pokes to the MOOSDB

Here we cover how to have a script of pre-arranged pokes to the [MOOSDB](#). This may be useful for a number of reasons besides debugging. The primary tool described here is the [uTimerScript](#) MOOS application. It is capable of (a) scripted pokes at pre-defined times after launch, (b) pokes having a poke-time specified to fall randomly within a specified interval, (c) pokes having numerical values falling with a uniformly random interval, and several other features including conditioning the running of the script based on other MOOS variables.

Where to get more information:

- [uTimerScript](http://oceanai.mit.edu/ivpman/apps/uTimerScript): <http://oceanai.mit.edu/ivpman/apps/uTimerScript>

Or simply:

```
$ uTimerscript --web
```

1.9.1 Basic uTimerScript Usage

[uTimerScript](#) is configured with its own block in the MOOS configuration file. The general format is below. The primary entries are the events themselves, defined by a MOOS variable, value, and time or time-range when the event is to occur. There are many options for configuring the script. These options are described in the documentation, but a quick look at the options can be seen by typing [uTimerScript --example](#) on the command line.

```
ProcessConfig = uTimerScript
{
  event = var=<MOOSVar>, val=<value>, time=<value>
  event = var=<MOOSVar>, val=<value>, time=<value>
  ...
  event = var=<MOOSVar>, val=<value>, time=<value>

  [OPTIONS]
}
```

1.9.2 A Simple Example with uTimerScript

The below mission file contains a **uTimerScript** script for repeatedly posting the variable COUNTER_A with values 1-10 in ascending order roughly once every half second. The last event in the script is posted at time chosen from a random five second interval.

Listing 1.2: A simple counter example with uTimerScript.

```
// (wget http://oceanai.mit.edu/2.680/examples/utscript.moos)
ServerHost = localhost
ServerPort = 9000
Community = alpha

ProcessConfig = ANTLER
{
    MSBetweenLaunches = 200

    Run = MOOSDB          @ NewConsole = false
    Run = uXMS             @ NewConsole = true
    Run = uTimerScript @ NewConsole = false
}

ProcessConfig = uXMS
{
    VAR          = COUNTER_A, DB_CLIENTS, DB_UPTIME
    COLOR_MAP     = COUNTER_A, red
    HISTORY_VAR   = COUNTER_A
}

ProcessConfig = uTimerScript
{
    paused = false

    event = var=COUNTER_A, val=1, time=0.5
    event = var=COUNTER_A, val=2, time=1.0
    event = var=COUNTER_A, val=3, time=1.5
    event = var=COUNTER_A, val=4, time=2.0
    event = var=COUNTER_A, val=5, time=2.5
    event = var=COUNTER_A, val=6, time=3.0
    event = var=COUNTER_A, val=7, time=3.5
    event = var=COUNTER_A, val=8, time=4.0
    event = var=COUNTER_A, val=9, time=4.5
    event = var=COUNTER_A, val=10, time=5:10

    reset_max = nolimit
    reset_time = all-posted
}
```

The mission may be launched from the command-line with:

```
$ pAntler utscript.moos
```

This should open a new console window for **uXMS** displaying the variables COUNTER_A variable repeatedly

incrementing from 1 to 10. Note that reaching 10 happens somewhere between 0.5 and 5.5 seconds after reaching 9.

1.9.3 Exercises

Your goals in this part are:

1. Create a copy of the example mission file in Listing 2 above and save it locally.
2. Launch the mission. It should open a `uXMS` window. Follow the progress of the counter script.

```
$ pAntler utscript.moos
```

3. Take a look at the `uTimerScript` documentation linked from the web page. In particular, Section 3 Script Flow Control. Configure the script such that it is paused when `uTimerScript` is launched. Launch the same mission and confirm that the script is initially not running. Then use `uPokeDB` to un-pause the script, and confirm it is running. Hint: to un-pause the script with `uPokeDB`, you'll need to know which variable to poke, and this also can be found in the `uTimerScript` documentation or by typing `uTimerScript -i` on the command line.
4. This is a bit of a `pAntler` exercise. Configure your mission to launch two versions of the script, the second version publishing to `COUNTER.B`. Note you will need two configuration blocks, each with a unique name. And you will need to launch `uTimerScript` twice within the Antler block, each with an alias. Hint: see the `pXRelay` example at the end of Lab 3.
5. Confirm your new mission launches and executes the two separate scripts and both counters are incrementing.
6. Configure the second script with a `condition` parameter. See Section 3.2 of the `uTimerScript` documentation. Use a condition such as "`condition = COUNTER_A > 5`". Re-launch your mission. Confirm that the second script is paused periodically based on the state of the first script.
7. Add the `pLogger` application to your mission. You will need to add a `pLogger` entry to your ANTLER configuration block, and add the following `pLogger` configuration block at the end of your file.

```
ProcessConfig = pLogger
{
  AsyncLog = true
  WildCardLogging = true
  WildCardOmitPattern = *_STATUS
}
```

Re-run the mission. Confirm that you see the `pLogger` application listed in the `DB_CLIENTS` variable in the `uXMS` scope.

8. Verify that a log file has been created. Since we didn't specify a name for the log file, by default it should be in a subdirectory of where you launched the mission, looking something like `MOOSLog-11.21.2024-----11.31.13/`. Enter the directory and confirm that you see an `.alog` file.
9. Take a look at the file by typing `more filename.alog`. Then take a look at the `COUNTER` variables using `aloggrep` (substituting of course the name of your `.alog` file:

```
$ aloggrep COUNTER_A COUNTER_B MOOSLog_11_23_2016_____11_31_13.alog
```

2 A Simple Example with pXRelay

pXRelay is a simple MOOS app designed solely to illustrate basic functions of a MOOS app. It registers for a single variable, and upon receiving mail for that variable, it publishes another variable incremented by 1. It provides a framework for illustrating a few other introductory topics.

2.1 Basic pXRelay Usage

pXRelay is configured with its own block in the MOOS configuration file. It is configured with (a) an incoming variable, the variable it will register for incoming mail, and (b) an outgoing variable, a variable it will post an incremented integer each time it receives mail on the incoming variable. The basic form is:

```
ProcessConfig = pXRelay
{
    outgoing_var = <MOOSVar>
    incoming_var = <MOOSVar>
}
```

2.2 A Simple Example with pXRelay

The below mission file contains a configuration for two instances of the **pXRelay** application. All MOOS apps must have a unique name to connect to the MOOSDB, so we launch them with an alias with **pAntler** using the `pXRelay_PEARs` alias for example. The two apps each register for what the other produces, and each produces what the other registers for.

Listing 2.3: Example Code.

```
0  // (wget http://oceanai.mit.edu/2.680/examples/xrelay.moos)
1  ServerHost = localhost
2  ServerPort = 9000
3  Community = alpha
4
5  ProcessConfig = ANTLER
6  {
7      MSBetweenLaunches = 200
8
9      Run = MOOSDB          @ NewConsole = false
10     Run = pXRelay         @ NewConsole = false ~pXRelay_PEARs
11     Run = pXRelay         @ NewConsole = false ~pXRelay_APPLES
12 }
13
14 ProcessConfig = pXRelay_APPLES
15 {
16     AppTick          = 10
17     CommsTick        = 10
18     incoming_var     = APPLES
19     outgoing_var     = PEARs
20 }
21
22 ProcessConfig = pXRelay_PEARs
```

```

23  {
24    AppTick      = 10
25    CommsTick    = 10
26    incoming_var = PEARS
27    outgoing_var = APPLES
28  }

```

Upon launch, the two `pXRelay` apps are in a stalemate, each waiting for the other to make the first posting. We can break this stalemate with `uPokeDB`:

```
$ uPokeDB xrelay.moos PEARS=1
```

This should get things going. Now it would be good to see it all running by launching a scope:

```
$ uXMS xrelay.moos --all --show=time
```

Your goals in this part are:

1. Create a copy of the example mission file shown in Listing 3 above and save it locally as `pxrelay.moos`. (hint: use `wget`!)
2. Launch the mission. Open up `uXMS` in another Terminal window with the parameters of your choosing. I recommend

```
$ uXMS pxrelay.moos --colorany=APPLES,PEARS --all
```

3. Kick off the activity by poking one of the `APPLES` or `PEARS` variables with an initial value. Confirm that things are working as they should.
4. Add `uTimerScript` to your mission file, with a simple script to kick off the `pXRelay` handshaking at some point after launch (say 10 secs), as an alternative way to kicking off the activity instead of `uPokeDB`. You'll need to add `uTimerScript` to your ANTLER configuration block, and add a simple script (a `uTimerScript` configuration block) to your `.moos` file.
5. Change your `uTimerScript` script to be the ascending counter script from Section 1.9, incrementing `COUNTER.A` 1 to 10. Configure it with `paused=false`, but add a condition to your script (`condition = APPLES == $(PEARS)`). Re-launch the revised mission. Since `APPLES` is equal to `PEARS` periodically, the condition will periodically be met. (You will still need to kick things off with either an initial poke from the command line, or re-use the script from the previous item as a second script to kick things off.)
6. Try changing the `AppTick` in one of the `pXRelay` configurations to 0.1. You should note that the script is now mostly in the state where its conditions are not met. Can you explain why?

3 First Experiments in Modifying MOOS App Code

Next we will take the first steps toward extending the MOOS-IvP code. This is done by creating your own "third-party" tree, with your own MOOS apps.

3.1 Download a Template Tree for Extending MOOS

In writing MOOS apps and later helm behaviors in this class, you'll need your own software folder. This folder will need a bit of structure and CMake files for compiling the source code. To make this a bit easier, a template for this is available. Follow the instructions in the separate [Version Control with Git](#) document alongside the other lab documents, linked from the Labs section on the course website. Follow these instructions before proceeding to [Section 3.2](#).

This folder has an example MOOS app and example helm behavior and the CMake files for building the code. We will use this as starting point for writing code in the class. And our first exercise below, involves trying to make some simple modifications of the example app.

The name of your tree should be `moos-ivp-janedoe`, where `janedoe` is your MIT Athena name. Hereafter we will simply refer to this tree generically as the `moos-ivp-extend` tree.

3.2 Building Your Tree

Verify that you can build your `moos-ivp-extend` tree by:

```
$ cd moos-ivp-extend
$ ./build.sh
```

It should add the executable, `pXRelayTest`, in `moos-ivp-extend/bin/`.

Add the above directory to your shell path. Replacing `extend` with your username in all of the above.

3.3 Exercises in Modifying the pXRelayTest Code

The `moos-ivp-extend` tree contains an app very similar to `pXRelay` from the `moos-ivp` tree. This app is call `pXRelayTest`. We will attempt to modify some of the behavior of the `pXRelay` application while putting our modifications in the new `moos-ivp-extend` tree, in `pXRelayTest`.

1. Take a look at the `pXRelayTest` source code. It is inside `moos-ivp-extend/src/pXRelayTest`. Look inside `Relayer.cpp`. Check out the `OnNewMail()` and `Iterate()` loops.
2. Can you modify this code to make it increment by ten instead of one?
3. Can you modify this code such that it takes *two* incoming variables, and does its thing if *either* of the two variables is written to?

Lab 3 - Introduction to MOOS Programming

2.680 Unmanned Marine Vehicle Autonomy, Sensing and Communications



February 10th, 2026

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1	Overview and Objectives	3
1.1	Preliminaries	3
1.2	MOOS, MOOS-IvP and Your Applications	3
1.3	More MOOS / MOOS-IvP Resources	4
1.4	The MOOS Application Structure	4
1.5	Handy Functions Defined on MOOSMsg (MOOS mail)	5
1.6	Assignment 1 (self check off)	7
2	Getting Started with Version Control	8
3	Build Your First MOOS App - An Odometry MOOS App	8
3.1	Generating a Template App and Augmenting the Build System	8
3.2	Assignment 2: (self check off) Verify Skeleton pOdometry has Built	9
3.3	Writing the Odometry MOOS App	9
3.4	Use Your pOdometry app in the Alder mission	10
3.5	Assignment 3 (check off) Verify Your pOdometry Works	12
3.6	Assignment 4 (check off) Convert pOdometry to an AppCasting MOOSApp	13
4	Some Hints for Debugging	14
4.1	Run the Logger	14
4.2	Debugging with Additional MOOS Publications	14
4.3	Debugging with Terminal Output, Method 1	15
4.4	Debugging with Terminal Output, Method 2 (Recommended)	15
5	Additional Exercises	16
5.1	Adding Terminal Output to your App for Debugging	16
5.2	Adding Run Time Checks for Steady Nav Information	16
5.3	Bonus: Configure the Staleness Threshold	16
5.4	Bonus: Configurable Units	17
6	Due Date and Grading Criteria	18
6.1	Due Date	18
6.2	Grading Criteria	18
6.3	Submitting Instructions for Lab 03	18

1 Overview and Objectives

In this lab we will produce our own MOOS applications. We begin by downloading an example application complete with its own build structure. We proceed by making our first simple MOOS app emphasizing the usage of the basic MOOS components. This is followed by a couple more complex application exercises.

This lab contains our first hand-in assignment. Although later in the class we will be working with partners, in this lab, all work should be done by individuals. When students post questions of general relevance, we will make that information available to everyone.

- The MOOS Application Structure (Iterate, OnNewMail, OnStartUp Methods)
- The MOOS Message Structure
- Getting Started with Software Version Control
- Downloading and Building the moos-ivp-extend tree
- Build Your First MOOS App - An Odometry App

1.1 Preliminaries

This lab does assume that you have a working MOOS-IvP tree checked out and installed on your computer. To verify this make sure that the following executables are built and findable in your shell path:

```
$ which MOOSDB
/Users/you/moos-ivp/bin/MOOSDB
$ which pHelmIvP
/Users/you/moos-ivp/bin/pHelmIvP
```

If unsuccessful with the above, return to the steps in Lab 1:

https://oceanai.mit.edu/ivpman/labs/machine_setup

1.2 MOOS, MOOS-IvP and Your Applications

In the previous lab we discussed the relationship between MOOS and MOOS-IvP. The MOOS tree is a body of software distributed as part of the MOOS-IvP tree as depicted in Figure 1. MOOS-IvP provides additional MOOS applications, including the IvP Helm behavior-based architecture, and has C++ build dependencies on the MOOS libraries. Today the focus is on building additional MOOS applications. Your MOOS apps will have a build dependency on the MOOS libraries, and you may choose to utilize libraries in the MOOS-IvP tree. We start by downloading the `moos-ivp-extend` tree which may be regarded as a template for extending the MOOS-IvP tree with apps and behaviors. This tree also includes a functioning build structure to ease the learning curve on C++ build issues for now. You likely have already downloaded and built this tree if you finished the last part of the previous lab.

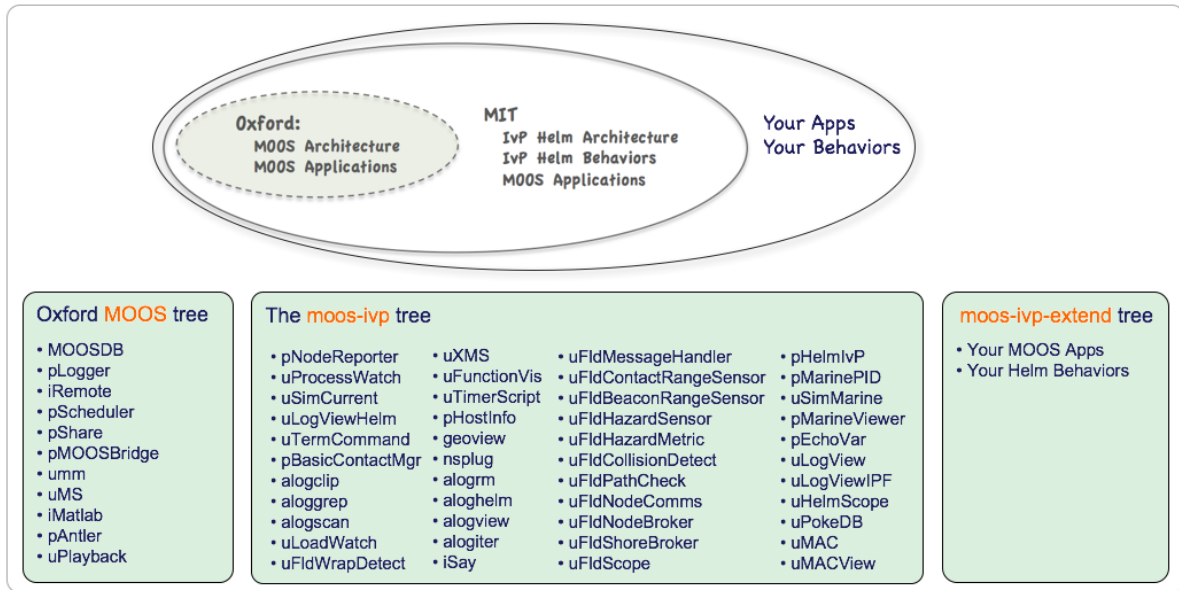


Figure 1: **Nested Repositories:** The MOOS-IvP tree contains the Oxford MOOS tree and additional modules from MIT including the Helm architecture, Helm behaviors and further MOOS applications. The set of modules may be expanded with user third-party applications or behaviors.

1.3 More MOOS / MOOS-IvP Resources

We will only just touch the MOOS programming basics today. A few further resources are worth mentioning for following up this lab with your own exploration.

- See the slides from the today's class which give a bit more background into MOOS and MOOS-IvP related to marine robotics.
https://oceanai.mit.edu/2.680/docs/2.680-03-moos_programming_2026.pdf
- The Programming with MOOS documentation.
<https://oceanai.mit.edu/2.680/docs/ProgrammingWithMOOS.pdf>
- The moos-ivp.org website documentation.
<https://oceanai.mit.edu/ivpman>

1.4 The MOOS Application Structure

The main idea explored today is the notion and structure of a MOOS application. We know from the last lab that MOOS apps publish, subscribe for, and handle mail passed from one application to another through the **MOOSDB**. In the last lab we worked with existing MOOS apps, only modifying their functionality through configuration options provided by the application author. Even without modifying the code of existing MOOS apps there are many ways to configure a system for unique domains. However, the real power of MOOS comes from the fact that no application is sacred. If you don't like what it does (or doesn't) do, you are free copy *and rename it*, and modify the code to your satisfaction. Or you can just build your own application from scratch. This is the focus of today's lab.

The key components to keep in mind in today's lab are shown in Figure 2 below. All MOOS apps begin by being a subclass of the MOOSApp superclass defined in the Oxford MOOS library. The primary work of the app developer is in writing the three functions shown in the figure.

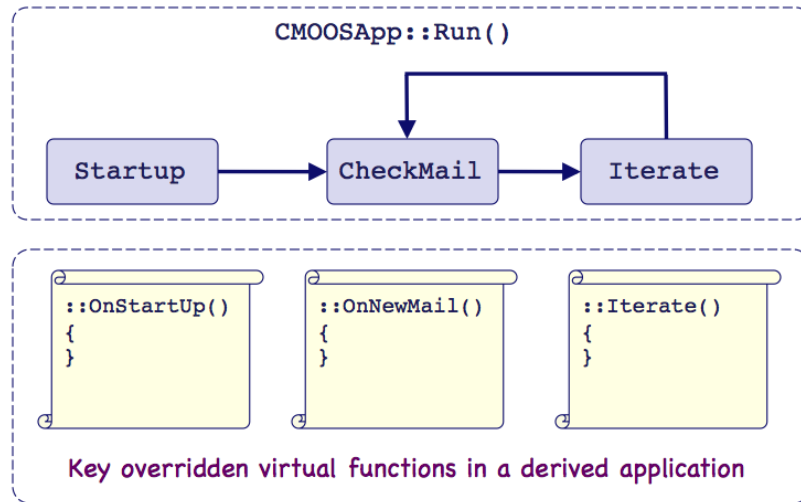


Figure 2: **The MOOSApp Key Functions:** All MOOS apps are a subclass of the MOOSApp superclass. Development mostly boils down to overriding the three functions below with the particulars of one's own liking.

Documentation Conventions

To help distinguish between MOOS variables, MOOS configuration parameters, and behavior configuration parameters, we will use the following conventions:

- MOOS variables are rendered in **green**, such as **IVPHELM.STATE**, as well as postings to the **MOOSDB**, such as **DEPLOY=true**.
- MOOS configuration parameters are rendered in **blue**, such as **AppTick=10** and **verbose=true**. Except for **AppTick** and **CommsTick**, configuration parameters are generally lower case.
- MOOS apps and executables generally, are rendered in magenta as with **MOOSDB** and **GenMOOSApp**.
- IvP Helm Behavior configuration parameters are rendered in **brown**, such as **priority=100** and **endflag=RETURN=true**.

This convention closely follows the convention used in the helm documentation.

1.5 Handy Functions Defined on MOOSMsg (MOOS mail)

The below methods are defined on an instance of **MOOSMsg**. Typically we are dealing with MOOS messages in the **OnNewMail()** loop of an application. Besides knowing the name of the variable, you may also want to know (a) what is its value, (b) by whom or which app was it posted, (c) when was it posted, (d) is its value of type string or double, and so on.

In our **pXRelayTest** example app, we only call **msg.GetKey()** in **Relayer.cpp**, but all the below methods are available for getting the message fields. (See today's lecture notes for a description of a

MOOS Message.) This information is probably in the MOOS documentation, but is repeated here for convenience.

```
// return the name of the message
std::string GetKey();

// return the name of the message (just another way)
std::string GetName();

// check data type is double
bool IsDouble();

// check data type is string
bool IsString();

// return time stamp of message
double GetTime();

// return double val of message
double GetDouble();

// return string value of message
std::string GetString();

// return the name of the process (as registered with the DB) which
// posted this notification
std::string GetSource();

// return the name of the MOOS community in which the originator lives
std::string GetCommunity();
```

Later on in this lab you will see that we provide a template for generating the skeleton code of a new MOOS app. This is done using the [GenMOOSApp](#) script. This skeleton code will generate the following `OnNewMail()` method:

```

bool MyApp::OnNewMail(MOOSMSG_LIST &NewMail)
{
    MOOSMSG_LIST::iterator p;
    for(p=NewMail.begin(); p!=NewMail.end(); p++) {
        CMOOSMsg &msg = *p;

    #if 0 // Keep these around just for template
        string key   = msg.GetKey();
        string comm  = msg.GetCommunity();
        double dval  = msg.GetDouble();
        string sval  = msg.GetString();
        string msrc  = msg.GetSource();
        double mtime = msg.GetTime();
        bool  mdbl   = msg.IsDouble();
        bool  mstr   = msg.IsString();
    #endif
    }
    return(true);
}

```

Note all the typical function calls you may want to make on an incoming MOOS message are right there, commented out to start with, but ready to use if you need them in your application. (Everything between the `if 0` and `endif` is ignored by the compiler, so these lines have the same effect as comment lines.)

1.6 Assignment 1 (self check off)

Verify that you have successfully downloaded and built the `moos-ivp-extend` tree, and that the binaries of this tree are findable in your shell path. If you haven't done this yet, see Section 2 for info on how to obtain the `moos-ivp-extend` tree from [git](#).

Verify with the following:

```

$ which pXRelayTest
$ /home/you/moos-ivp-you/bin/pXRelayTest

```

If the `which` function returns no value, then either the binary `pXRelayTest` was not built or you haven't successfully augmented your shell path. First check to ensure it was built by examining the contents of `moos-ivp-you/bin/`. If `pXRelayTest` is there, then double check that you have properly augmented your shell path. If need be, return to the help page:

https://oceanai.mit.edu/ivpman/help/cmdline_augment_shell_path

You can also display your shell's current path with:

```

$ echo $PATH

```

If you don't see `/home/you/moos-ivp-you/ivp/bin` in the colon-separated list of locations constituting your shell path, then you are not done.

2 Getting Started with Version Control

In Lab 3, a supplement document distributed for getting started with `git`, and obtaining a copy of your `moos-ivp-extend` tree. This document is online here:

<https://oceanai.mit.edu/ivpman/labs/git>

At this stage you should have few steps under your belt:

- Created a GitHub account
- Created a new `moos-ivp-extend` repo by importing the generally provided `moos-ivp-extend` repository.
- Provided collaborator access to the TAs
- Confirm you know how to modify a file in your tree and commit the changes
- Confirm you can add a file or directory to your tree and commit changes.

If the above steps are not complete, you should make sure they are complete before moving on to the next section.

3 Build Your First MOOS App - An Odometry MOOS App

The focus of this section is on building your first MOOS App. Your goals are:

1. Learn how to generate a MOOS application from scratch using a template-generating script.
2. Learn how to add the new MOOS application to the build system.
3. Write your new MOOS app, `pOdometry`, to calculate the total distance traveled by the vehicle.
4. Test your MOOS app by using it in the Alder example mission to have the vehicle return after traveling 50 meters.
5. Convert your MOOS app into an AppCasting MOOS App.

3.1 Generating a Template App and Augmenting the Build System

Your goal in this part is to generate a new application by using a shell script to generate a full application template. The resulting template should be buildable immediately without further modification, but will essentially do nothing meaningful until you add your own functionality.

3.1.1 Generate the MOOS App

To generate the new MOOS app, in a terminal window change directories to `moos-ivp-extend/src/`. From here you should be able to invoke the below command:

```
$ GenMOOSApp_AppCasting Odometry p "Jane Doe"
```

The above script, `GenMOOSApp_AppCasting`, should be in your shell path under `moos-ivp/scripts/`. You may need add this directory to your shell path if you haven't done so already. The above script

invocation builds a new app called `pOdometry`. The third argument is your name. You can type `GenMOOSApp-AppCasting -h` in the future to remind yourself.

3.1.2 Add Your Name

Always, always, always put your name at the top of your source code files! This is a good practice in general, but even more so in a class setting, or an open source environment where people are sharing code. It wouldn't hurt to add the date and organization information too.

3.1.3 Add Your New Application to the Build System

Add your new app to the build system. Edit the file `moos-ivp-extend/src/CMakeLists.txt` and look for the line referring to `pXRelayTest`. Copy that as an example for your new app.

```
#=====
# List the subdirectories to build...
#=====
ADD_SUBDIRECTORY(lib_behaviors-test)
ADD_SUBDIRECTORY(pXRelayTest)
ADD_SUBDIRECTORY(pExampleApp)
ADD_SUBDIRECTORY(pOdometry)                <-- Add your line here
```

3.2 Assignment 2: (self check off) Verify Skeleton `pOdometry` has Built

Go back and re-run your build script. Check to see that `pOdometry` has been built, and is in your bin directory (e.g., `moos-ivp-you/bin`).

Verify that `pOdometry` is in your shell path with:

```
$ which pOdometry
$ /home/you/moos-ivp-you/bin/pOdometry
```

If the `which` function returns no value, then either the binary `pXRelayTest` was not built or you haven't successfully augmented your shell path. See above in Section 1.6 for tips on how to remedy the problem.

3.3 Writing the Odometry MOOS App

The goal of this step is to build a new simple MOOS application that calculates vehicle odometry. It will:

1. Register for the `NAV_X` and `NAV_Y` position of the vehicle.
2. Repeatedly read in the `NAV_X` and `NAV_Y` position of the vehicle.
3. Update the total distance traveled by calculating the distance between the present position and the previous position.
4. Post the odometry information in the MOOS variable `ODOMETRY_DIST`.

5. The posted odometry distance should be consistent regardless of the AppTick of the `pOdometry` app. You can check this by changing the AppTick of your app while leaving the AppTick of `uSimMarineV22` unchanged.

We will do this in your newly checked out `moos-ivp-extend` tree. Begin as follows:

- Edit `Odometry.h` to have at least the six member variables:

```
bool    m_first_reading;
double  m_current_x;
double  m_current_y;
double  m_previous_x;
double  m_previous_y;
double  m_total_distance;
```

You may of course use further member variables if you see the need.

- Edit `Odometry.cpp` to initialize the above member variables, in the class constructor.
- Edit `Odometry.cpp` to register for `NAV_X` and `NAV_Y`.
- Edit the `Odometry::OnNewMail()` method to handle the updated navigation position.
- Edit the `Odometry::Iterate()` method to calculate the new distance. Handle the special case of the first navigation measurement. Post the total distance to the MOOS variable `ODOMETRY_DIST`.
- Verify that everything builds...

There is one common "gotcha" in this lab. Keep in mind that the `NAV_X` and the `NAV_Y` variables are being published by the `uSimMarineV22` application. This application is running at 10Hz and producing on average 10 incoming mail messages for your `pOdometry` app. If your `pOdometry` app is running at the default rate of 4Hz, then on average your app will have 2-3 incoming mail messages on each iteration. If you find that your odometry calculations only reflect about half of the actual distance traveled, reconsider how you are calculating the length of new legs.

3.4 Use Your `pOdometry` app in the Alder mission

The first goal is to just make sure it runs and posts the correct information. The second goal is to use the information to alter the mission behavior.

NOTE: Depending on when you cloned the `moos-ivp-extend` tree, you may need to ensure that all references to `uSimMarine` are replaced with `uSimMarineV22` and all references to `pMarinePID` are replaced with `pMarinePIDV22`, in the `alder.moos` file.

3.4.1 Running the Un-modified Alder Mission

First make sure that you can run the Alder mission. Try it:

```
$ cd moos-ivp-extend/missions/alder
$ pAntler --MOOSTimeWarp=10 alder.moos
```


It should look something like the video posted at:

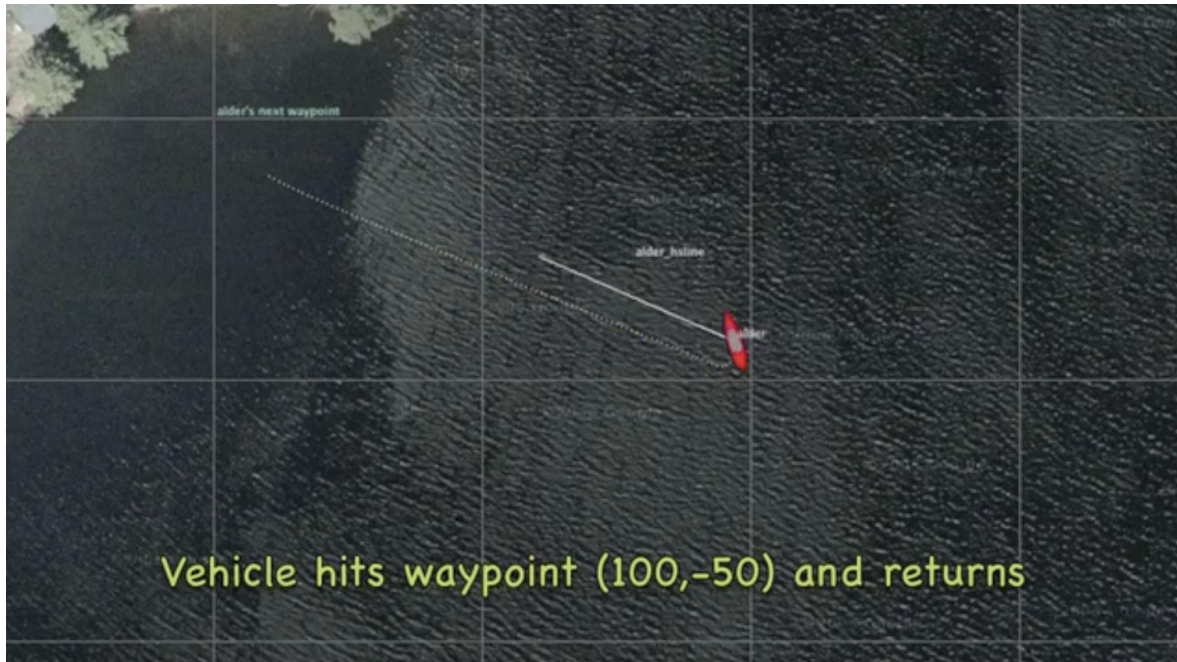


Figure 3: The Alder mission. The vehicle travels to point (100,-50) and returns home.

3.4.2 Run the Alder Mission with pOdometry

Next we'll add the `pOdometry` application to the mission and just verify it is generating the right output. Do the following steps:

- Modify the Alder mission (`moos-ivp-extend/missions/alder/alder.moos`) to include `pOdometry` at launch time by editing the `Antler` block in the `alder.moos` file.
- Add `ODOMETRY_DIST` to the scope list in `pMarineViewer`. See the `pMarineViewer` documentation to see how to do this. It should look something like:

```
ProcessConfig = pMarineViewer
{
  AppTick      = 4
  CommsTick    = 4

  ...
  scope = ODOMETRY_DIST    // <-- Add this line
  ...
}
```

- Re-launch the mission. Now you should be able to see the odometry distance in the scope field at the bottom of `pMarineViewer`. Does it look correct?

3.4.3 Modify the Alder Mission Using pOdometry

In the next step, we will modify the alder mission as follows. Instead of transiting to the waypoint specified in the waypoint behavior, the vehicle will return home after it has transited 50 meters. We haven't covered the Helm yet in class, so some of this we'll take on faith for now.

Take a look inside `alder.bhv`. This is where the helm is configured. The mission is comprised of two instances of a waypoint behavior. The first transits to a given point and is called `waypt_to_point`. It has two conditions, we'll add a third:

```
//-----  
Behavior = BHV_SimpleWaypoint  
{  
  name      = waypt_to_point  
  pwt       = 100  
  condition = RETURN = false  
  condition = DEPLOY = true  
  condition = (ODOMETRY_DIST < 50)      // <-- Add this line  
  
  endflag   = RETURN = true  
  
  speed     = 2.0    // meters per second  
  radius    = 8.0  
  ptx       = 100  
  pty       = -50  
}
```

The second waypoint behavior is designed to return after the first one completes. It also has two conditions:

```
condition = RETURN = true  
condition = DEPLOY = true
```

We'll change that to:

```
condition = (RETURN = true) or (ODOMETRY_DIST >= 50)  
condition = DEPLOY = true
```

3.5 Assignment 3 (check off) Verify Your pOdometry Works

After the above modifications, re-launch the mission. Demonstrate to one of the TAs that your `pOdometry` works. It should look something like:



Figure 4: The Alder mission with odometry. The vehicle begins to travel to point (100,-50), but returns home early based on odometry distance.

3.6 Assignment 4 (check off) Convert pOdometry to an AppCasting MOOSApp

The last step will be to convert your `pOdometry` application to be appcast enabled. Follow the instructions from today's lecture notes, or read the documentation for enabling appcasting:

https://oceanai.mit.edu/ivpman/appcast_enable

If you used the `GenMOOSApp_AppCasting` script to make your initial folder for `pOdometry`, then your app is already mostly AppCast enabled. You just need to implement the AppCasting output. The output of your appcast report (in the `buildReport()` function) should simply show the total distance traveled.

4 Some Hints for Debugging

There are a few common methods for debugging a MOOS App, and we'll have more to say about this during the semester. But here are a few methods. The third method described below is a new feature added only in the last six months and is probably your most powerful option.

4.1 Run the Logger

If your Alder mission is not yet including `pLogger`, you will need to add it to your mission for these steps.

To add `pLogger` to your mission, add it the Antler config block with the additional line:

```
Run = pLogger @ NewConsole = false
```

You will also need a configuration block for `pLogger` in the same mission file, e.g., `alder.moos`. The below block should work fine:

```
ProcessConfig = pLogger
{
  AppTick      = 8
  CommsTick    = 8

  AsyncLog     = true
  WildCardLogging = true
}
```

4.2 Debugging with Additional MOOS Publications

In this lab, the primary output of your app is the variable `ODOMETRY_DIST`. You are free to publish as many other variables as you like, to perhaps reveal any intermediate calculations, or other information. For example you can publish `LEG_DIST` if you are calculating the distance one leg at a time.

To utilize this method, you will need to either (a) scope on this additional variable while the mission is running, or (b) examine the variable from the log file after the mission is complete. The latter method gives you more time to ponder the values. To examine the log file, you can use `allogrep`, along the lines of:

```
$ allogrep logfile.alog ODOMETRY_DIST LEG_DIST
```

Or you can open `alogview` and scope on the above two variables using the `VarHist` pull-down menu. See the documentation for `alogview`. Either point your browser to:

<https://oceanai.mit.edu/ivpman/apps/alogview>

Or, on the command line, just type the below, and your browser should take you there.

```
$ alogview --web
```

4.3 Debugging with Terminal Output, Method 1

Another powerful method for debugging is to add `cout` statements to your code, to show intermediate values. So, for example, instead of publishing `LEG_DIST` as a MOOS variable, you can instead add something like this to your code:

```
double leg_dist = ...  
cout << "Leg dist: " << leg_dist << endl;
```

To see this output, you would need to launch your app with a terminal window open, OR remove your app from the Antler block and run it separately from another terminal window. So in the first terminal window:

```
$ ./launch.sh
```

In the second terminal window:

```
$ pOdometry file.moos
```

Make sure you use the same `.moos` file in both launches. And note that if you launch your mission with a time warp, `Antler` will generate a `file.moos++` mission file with the proper time warp. You would then need to launch your app with the same `.moos++` file to get the same time warp.

4.4 Debugging with Terminal Output, Method 2 (Recommended)

If you have successfully created your app as an AppCasting MOOS App, then there is another simpler option for debugging with terminal output.

As above, inject your code with `cout` statements to produce what you would like. But rather than running your app in a separate Terminal window, simply add the below line to the configuration block for your app:

```
app_logging = log
```

This will log all the terminal output from your app in the `alog` file. Then you can view your terminal output by launching `alogview`. Once this is launched, select `AppLog` from the pull-down menu, select the vehicle name, and select your app from the list. You should then be able to see all your terminal output.

If you are trying this debugging method and have issues, please come find me. This is a new feature and not yet documented in the `alogview` documentation.

5 Additional Exercises

5.1 Adding Terminal Output to your App for Debugging

At some point you will need to debug a faulty app. A powerful tool for doing this is to generate terminal output (`cout` commands). In this exercise we will generate some simple output and show how we can view this output *after* the mission has completed using the `alogview` tool.

- Make sure your app is AppCast enabled if it is not already.
- Add terminal output in your app to show each new `NAV_X` and `NAV_Y` received, e.g.,
`cout << "nav_x: " << value << endl;`
- Add terminal output in your app to show each new value of `ODOMETRY_DIST` calculated.
- Enable app logging by setting `app_logging=log` in the `pOdometry` configuration block of your mission file.
- Run your mission to completion.
- Find the alog file and launch `alogview` on this alog file.
- Select AppLog from the pull-down menu and confirm that you can see your terminal output.
- Show a TA.

Note that some of this functionality could also be achieved by generating AppCasting output. This is not a replacement for that. One advantage of AppLogging is that terminal output can be captured regardless of the level of code being executed. The `cout` call may be used in code that is called by your app several levels down.

5.2 Adding Run Time Checks for Steady Nav Information

Another powerful tool for developing robust apps is the use of run time err checking. AppCasting MOOS Apps support this with the `reportRunWarning()` function. In this exercise:

- Add a timestamp (a new member variable) to note when the latest `NAV_X` or `NAV_Y` was received.
- If an interval of 10 seconds or more passes, without receiving new NAV info, generate a run warning.
- Retract this warning when or if the NAV info resumes.
- To test this, run the Alder mission as normal. In another terminal window, type `killall uSimMarineV22`. This will halt the NAV info. Verify the run warning appears in your Odometry app.
- Then, also in a separate terminal window in your Alder mission, restart the simulator with `uSimMarineV22 alder.moos`.
- Show a TA.

5.3 Bonus: Configure the Staleness Threshold

To gain some experience with configuration parameters:

- Add a configuration parameter to set the staleness threshold to something else besides the above default of 10 seconds.

- You will need to make this value a variable in your MOOS App instead of the the hard-coded "10" as before.
- Perform a check on the configuration parameter that it is a number greater than zero. If a user provides a faulty number in the mission file, produce a configuration warning.
- Show a TA.

5.4 Bonus: Configurable Units

Parameters that may be configured at *launch-time* are sometimes also configurable at *run-time*. There are many reasons for this. The two most-common reasons are (1) a parameter setting *augments* a set of parameters given at launch-time, with additional components that become known at run-time. (2) a parameter setting may involve the verbosity of output, perhaps in toggling on/off the publication of a MOOS variable providing debugging output. The need for debugging verbosity may change during run-time.

In this last bonus exercise, the task is to augment the `pOdometry` app with the ability to configure an additional publication of odometry distance in any arbitrary units provided either at configuration time or at run-time.

6 Due Date and Grading Criteria

6.1 Due Date

All MOOS apps described in this lab are due during lab Thursday February 12th, 2026. Apps will be tested during lab, and source code is due to the graders at the end of the lab.

6.2 Grading Criteria

Grading will be based on

1. Implementation Credit (60%)
2. Code Organization (40%)

Full credit will given be based on whether or not you have met the given specs.

Code organization includes the issue of commenting your code, and choosing a robust code structure, e.g., properly initializing variables etc. Beginners to C++ programming can expect a healthy amount of constructive criticism in the first assignments. See the 9th and 10th C++ labs on the website: "C++ Coding Guidelines" and "C++ Coding Style Guidelines".

Unfinished assignments will be tested and accepted in the next lab. A 5% deduction will be applied for each lab that goes by unfinished.

6.3 Submitting Instructions for Lab 03

Lab 03 assignments should be submitted using the version control mechanism described earlier. Your directory format should have the naming and file organization as indicated below:

```
moos-ivp-yourname/  
  src/  
    p0dometry  
  
  missions/  
    alder/  
      alder_lab3.moos  
      alder_lab3.bhv  
  
  lib/           (should be empty)  
  bin/           (should be empty)  
  build/         (should be empty)  
  build.sh       (should be unchanged)  
  CMakeLists.txt (should be unchanged)
```